

DOCUMENTACAO TECNICA

SaaS de Cobrancas Recorrentes

Boleto . PIX . NF-e . Portal Multi-Tenant

Engenharia Reversa + Especificacao Completa de Desenvolvimento

Versao 1.0 | Maio 2026
Confidencial -- uso interno

1. Resumo Executivo

Este documento consolida três perspectivas sobre o produto SaaS de cobranças:

- Engenharia reversa do código existente (cobrança-saas-api) com análise de baixo nível de cada módulo, banco de dados, contratos de API e lacunas técnicas.
- Visão de Product Owner com epícos, histórias de usuário, critérios de aceite e roadmap priorizado.
- Especificação completa de desenvolvimento para o produto final: backend, frontend, integrações com gateways (Boleto, PIX), banco de dados completo, n8n, NF-e e DevOps.

O contexto de negócio é o de uma empresa com recebimentos recorrentes de múltiplos clientes via Boleto e PIX, que precisa de uma plataforma que centralize emissão, controle de status, regua de cobrança automatizada e emissão de Nota Fiscal de Serviço (NFS-e) após pagamento confirmado.

2. Engenharia Reversa -- Estado Atual doCodigo

2.1 Visao Geral da Arquitetura

O repositório cobrança-saas-api é um monolito modular em Node.js/TypeScript extraído de um sistema maior (Projeto_EmissaoNF). A estrutura segue princípios de Domain-Driven Design com separação em camadas por módulo:

- domain -- entidades e tipos puros (ex: Charge, CanonicalChargeStatus)
- application -- casos de uso e regras de negócio (ex: createCharge, chargeStatusTransition)
- infrastructure -- repositórios com SQL raw via driver pg do PostgreSQL
- interfaces/http -- roteadores Express com middlewares de autenticação, tenant e RBAC
- platform -- infraestrutura transversal: pool de conexões, health checks, RLS, middlewares

2.2 Mapa de Modulos

Modulo	Rota Base	Responsabilidade	Status
identity-access	/v1/auth	JWT sign/verify, login bcrypt, mock dev	Implementado
billing-core	/v1/billing	CRUD cobranças, máquina de estados, idempotência	Implementado
inbox	/v1/inbox	Fila de webhooks, dedup por external_event_id, job process-pending	Implementado
portal-read	/v1/portal	Auth portal, clientes CRUD, cobranças portal, notas fiscais, paginação cursor	Parcial
tenant-provisioning	/v1/tenants	Criar escritório (tenant), vínculo billing_tenant_link	Parcial -- 45% coverage
platform/health	/health, /health/ready	Liveness e readiness com probe Postgres + schema mínimo	Implementado

2.3 Banco de Dados -- Schema Atual

2.3.1 Schema public (RLS ativo via app.tenant_id)

Tabela	Colunas-chave	RLS	Observação
tenants	id UUID, slug TEXT UNIQUE, name, status (active/suspended/trial), created_at	Não	Catálogo; resolução por slug/uuid antes do SET LOCAL
charges	id, tenant_id FK, reference, idempotency_key, amount NUMERIC(14,2), due_date DATE, canonical_status, provider, provider_charge_id, metadata JSONB	SIM -- policy por app.tenant_id	UNIQUE (tenant_id, reference) e (tenant_id, idempotency_key)
webhook_inbox	id, tenant_id, source, external_event_id, payload JSONB, correlation_id, processed_at, last_error, processing_attempts INT	SIM	UNIQUE (tenant_id, external_event_id) para dedup

2.3.2 Schema portal

Tabela/View	Colunas-chave	Observacao
portal.app_user	id UUID, email TEXT UNIQUE, full_name, password_hash (migracao 011), created_at	Usuarios do portal; bcrypt na coluna password_hash
portal.membership	id, app_user_id FK, tenant_id TEXT, role (admin_escritorio/operador/cliente_cnpj), cpf_cnpj_cliente	cliente_cnpj DEVE ter cpf_cnpj_cliente; admin/op NAO podem ter
portal.vw_notas_fiscais_resumo	JOIN automacao.notas_fiscais + automacao.tenants por tenant_id	View de leitura; DEPENDE do schema automacao do monolito origem -- risco critico

RISCO CRITICO: O schema automacao (notas_fiscais, tenants) pertence ao monolito Projeto_EmissaoNF e NAO esta incluido no repositorio entregue. O produto nao funciona de forma autonoma sem esse banco. O roadmap desta documentacao trata o desacoplamento como Fase 0.

2.4 Maquina de Estados de Cobrancas (100% cobertura de testes)

Estado	Significado	Transicoes de saida permitidas
rascunho	Criada mas nao emitida ao provedor	emitida, cancelada
emitida	Enviada ao gateway de pagamento	enviada, cancelada, erro_emissao
enviada	Comunicada ao devedor	pendente_pagamento, vencida, cancelada
pendente_pagamento	Acessada/visualizada pelo devedor	paga, vencida, cancelada
paga	Pagamento confirmado pelo gateway	-- terminal
vencida	Data ultrapassada sem pagamento	cancelada (ou nova cobranca gerada)
cancelada	Cancelada manual ou por regra	-- terminal
erro_emissao	Falha na emissao no gateway	emitida (retry), cancelada

2.5 Contratos HTTP -- Endpoints Implementados

2.5.1 Autenticacao do Portal

Endpoint	Metodo	Body	Resposta	Auth
POST /v1/portal/auth/login	POST	{ email, tenant_id, password }	{ token: JWT, user_id, tenant_id }	Publica
POST /v1/portal/auth/token/mock	POST	{ sub, tid, roles }	{ token: JWT }	Apenas dev (ENABLE MOCK_AUTH=true)

2.5.2 Clientes do Portal

Endpoint	Metodo	Descrição	Auth
GET /v1/portal/clientes	GET	Lista paginada (cursor keyset). Query: limit (1-200), cursor	JWT portal + membership
POST /v1/portal/clientes	POST	Cria cliente. Valida CPF/CNPJ + email. 409 se documento duplicado no tenant	admin_escritorio ou operador
GET /v1/portal/clientes/:id	GET	Detalhe do cliente	JWT portal
PATCH /v1/portal/clientes/:id	PATCH	Atualiza nome, email, opt-in. Documento (CPF/CNPJ) e imutável	admin_escritorio ou operador

2.5.3 Cobrancas do Portal

Endpoint	Metodo	Descrição	Auth
GET /v1/portal/cobrancas	GET	Lista paginada com filtro de status	JWT portal
POST /v1/portal/cobrancas	POST	Cria cobrança. Exige billing_tenant_link. Idempotencia por key	admin_escritorio ou operador
PATCH /v1/portal/cobrancas/:id	PATCH	Edita valor, vencimento, metadata. Bloqueado se paga ou cancelada	admin_escritorio ou operador
GET /v1/portal/clientes/:id/cobrancas	GET	Cobrancas paginadas por cliente	JWT portal

2.5.4 Webhooks e Core

Endpoint	Metodo	Descrição	Auth
POST /v1/inbox/webhooks	POST	Recebe evento externo. Header X-Webhook-Secret obrigatório em prod. Salva na fila com dedup	Secret header
POST /v1/inbox/webhooks/process-pending	POST	Processa fila pendente (job/cron). Tenant-scoped	JWT admin
POST /v1/billing/charges	POST	Cria cobrança no core (canal interno/automacao)	JWT tenant
GET /v1/billing/charges	GET	Lista cobranças do tenant (canal interno)	JWT tenant
GET /health	GET	{ status: ok, service }	Publica
GET /health/ready	GET	Probe Postgres + schema mínimo. 503 se falhar	Publica

2.6 Cobertura de Testes (dados reais do coverage-summary.json)

Arquivo	Linhas	Funções	Branches	Avaliação
charge-status-transition.ts	100%	100%	100%	Excelente
create-charge.ts	100%	100%	100%	Excelente
portal-password.ts	100%	100%	100%	Excelente
patch-portal-charge.ts	100%	100%	100%	Excelente
parse-webhook-charge-payload.ts	92.7%	100%	85.7%	Bom

Arquivo	Linhas	Funcoes	Branches	Avaliacao
portal-cliente-input.ts	93.8%	100%	84.9%	Bom
portal-list-cursor.ts	87.0%	100%	62.5%	Regular -- cursor branches descobertos
br-cpf-cnpj.ts	75.6%	100%	64.5%	Regular -- edge cases CPF/CNPJ
provision-public-tenant.ts	45.1%	66.7%	61.9%	RUIM -- ponto cego critico
TOTAL	83.9%	97.0%	76.9%	Acima da meta interna de 82%

2.7 Lacunas Técnicas Identificadas

2.7.1 Criticas -- bloqueiam o produto

- Nenhum gateway de pagamento integrado -- o sistema recebe webhooks mas não emite boletos nem cobra via PIX.
- Integração n8n ausente -- regra de cobrança, envio de WhatsApp e lembretes dependem de workflows que não estão no repositório.
- NF-e permanece no monolito origem -- produto não é autônomo para emissão fiscal.
- Sem envio de e-mail ou notificações de qualquer tipo.
- Sem billing do próprio SaaS -- não há lógica de planos e cobrança dos escritórios pelo uso da plataforma.

2.7.2 Importantes -- degradam produção

- Sem rate limiting em nenhuma rota -- /v1/portal/auth/login vulnerável a brute force.
- Sem Dockerfile ou docker-compose -- deploy manual sem padronização.
- Sem monitoramento de erros (Sentry ou similar).
- Arquivo .env com credenciais incluído no zip entregue -- prática de segurança inadequada.

2.7.3 Frontend (apps/portal-web)

- RelatoriosPage: placeholder -- tela em branco com mensagem "em breve".
- NotasFiscaisPage: apenas listagem sem ações.
- Sem design system -- componentes mínimos sem biblioteca UI definida.
- Sem testes E2E (Playwright, Cypress).

3. Visão de Produto -- Product Owner

3.1 Problema de Negócio

Empresas com carteira de clientes recorrentes (escritórios contábeis, consultorias, prestadores de serviço B2B) enfrentam três dores principais:

- Controle manual de cobranças via planilha -- alto risco de inadimplência por falta de acompanhamento sistemático.
- Processo de cobrança reativo -- o follow-up acontece após o vencimento, não antes.
- Fragmentação de ferramentas -- boleto gerado em um sistema, controle de pagamento em outro, NF-e em um terceiro.

3.2 Proposta de Valor

Uma plataforma única que centraliza: emissão de Boleto e PIX, regua de cobrança automatizada (pre e pós-vencimento via WhatsApp e e-mail), controle de status em tempo real via webhook do gateway, emissão automática de NFS-e após pagamento confirmado, e portal de autoatendimento para o cliente final consultar e baixar cobranças e notas fiscais.

3.3 Personas

Persona	Perfil	Objetivo principal	Dor atual
Admin do Escritório	Gestor do escritório. Papel admin_escritorio no portal.	Visualizar inadimplência, configurar regua, gerar relatórios.	Passa horas em planilha toda semana para controlar quem pagou.
Operador	Colaborador que processa cobranças. Papel operador.	Cadastrar clientes, emitir cobranças, registrar ocorrências.	Precisa acessar múltiplos sistemas para completar uma emissão.
Cliente Final (CNPJ)	Empresa devedora. Papel cliente_cnpj no portal.	Ver histórico, baixar boletos e NFs, pagar com PIX.	Recebe cobrança por e-mail mas não tem onde consultar histórico.
Proprietário da Plataforma	Empresa dona do SaaS.	Monitorar MRR, inadimplência de planos, crescimento de uso.	Sem visibilidade consolidada do negócio SaaS.

3.4 Epícos e Histórias de Usuário

Epíco 1 -- Gestão de Escritórios e Usuários

1. Como Admin, quero criar um escritório na plataforma informando CNPJ, razão social e plano, para que meu time possa começar a operar. CA: POST /v1/tenants/provision cria tenant com status=trial. E-mail de boas-vindas em até 60 segundos.
2. Como Admin, quero convidar colaboradores com papel de Operador. CA: link de convite com validade de 72h; papel operador confirmado após sete dias de senha.
3. Como Admin, quero configurar os dados fiscais do escritório (CNPJ emissor, série, regime tributário). CA: validação de CNPJ via Receita Federal; salvo em escritorio_config.

Epico 2 -- Cadastro e Gestao de Clientes

4. Como Operador, quero cadastrar um cliente com CPF ou CNPJ, nome e e-mail. CA: validacao de digito verificador no backend; 409 se documento ja existir no tenant.
5. Como Operador, quero editar nome e e-mail sem alterar o documento (CPF/CNPJ imutavel). CA: PATCH rejeita campo documento com 422.
6. Como Admin, quero ver historico de cobrancas de cada cliente para identificar devedores recorrentes. CA: tela de detalhe lista cobrancas com filtro de status e paginacao por cursor.

Epico 3 -- Emissao e Controle de Cobrancas

7. Como Operador, quero emitir uma cobranca escolhendo entre Boleto ou PIX, informando valor, vencimento e descricao. CA: cobranca criada com status=rascunho; emissao no gateway em ate 5 segundos via job assincrono; transicao para emitida confirmada por webhook de retorno.
8. Como Sistema, quero garantir que cobrancas duplicadas (mesmo idempotency_key) retornem 200 com idempotent: true sem duplicar o registro.
9. Como Admin, quero cancelar uma cobranca nao paga. CA: PATCH com status=cancelada bloqueado se paga; cancelamento no gateway confirmado por webhook.
10. Como Operador, quero corrigir valor ou vencimento de uma cobranca antes do pagamento. CA: PATCH bloqueado se canonical_status for paga ou cancelada.

Epico 4 -- Regua de Cobranca Automatizada

11. Como Admin, quero configurar regua que envie lembretes por WhatsApp e e-mail em D-3, D-1, D+0, D+3 e D+7 do vencimento. CA: workflows n8n associados ao tenant; disparos registrados em communication_events.
12. Como Sistema, quero que a regua pare quando pagamento for confirmado. CA: evento pago via webhook cancela jobs pendentes na fila do tenant.

Epico 5 -- PIX e Boleto (Integracoes com Gateways)

13. Como Sistema, quero gerar QR Code PIX dinamico pelo padrao EMV do Banco Central. CA: QR Code gerado em ate 2 segundos; payload EMV valido; link de pagamento funcionando em qualquer app bancario.
14. Como Sistema, quero gerar boletos bancarios registrados FEBRABAN. CA: boleto gerado via Asaas/Pagar.me; PDF disponivel para download em ate 10 segundos.
15. Como Sistema, quero receber confirmacao de pagamento via webhook do gateway em ate 5 minutos apos o credito. CA: webhook processa em background; canonical_status=paga; evento de NFS-e disparado.

Epico 6 -- Nota Fiscal Eletronica (NFS-e)

16. Como Sistema, quero emitir NFS-e automaticamente quando canonical_status mudar para paga. CA: emissao disparada em ate 30 segundos apos confirmacao; numero NFS-e e codigo de verificacao salvos na cobranca.
17. Como Cliente Final, quero baixar minha NFS-e pelo portal. CA: PDF disponivel na tela de detalhe com botao de download.

Epico 7 -- Portal do Cliente Final

18. Como Cliente Final, quero acessar o portal com meu e-mail e senha para ver cobranças em aberto. CA: login com papel cliente_cnpj; ve apenas cobranças do seu CNPJ.
19. Como Cliente Final, quero pagar via PIX diretamente no portal. CA: QR Code exibido na tela de detalhe; polling de status a cada 5 segundos por até 5 minutos.

Epico 8 -- Relatorios e Dashboard

20. Como Admin, quero dashboard com: total recebido no mes, em aberto, taxa de inadimplencia e proximos vencimentos nos 7 dias.
21. Como Admin, quero exportar relatorio de cobranças em CSV com filtros de periodo e cliente. CA: geracao em até 30 segundos; download direto no browser.

Epico 9 -- Billing do SaaS

22. Como Proprietario da Plataforma, quero cobrar os escritorios mensalmente por plano (basico/profissional/enterprise) com cobrança automatica via cartão ou boleto. CA: trial de 14 dias; downgrade automatico para read-only se assinatura vencer.

3.5 Definition of Done (DoD) por PR

- npm run build sem erros TypeScript.
- Cobertura de testes $\geq 85\%$ no escopo application/domain do PR.
- Novos endpoints documentados em API_CONTRATO_E_SMOKE.md antes do merge.
- Integrações com banco: casos de integração ou extensão da bateria funcional.
- Variáveis de ambiente novas documentadas no .env.example.
- Migrações versionadas com nome descritivo (NNN_descricao.sql).
- Code review aprovado por pelo menos 1 par com checklist de segurança.
- CI verde no GitHub Actions (build + testes + migrate + readiness + integração).

4. Arquitetura Tecnica Completa

4.1 Stack Tecnologico Definido

Camada	Tecnologia	Versao	Justificativa
Runtime	Node.js	20 LTS	Compativel com codigo existente; suporte ate 2026
Linguagem	TypeScript	5.7+	Type safety; ja em uso
Framework HTTP	Express	4.x	Ja em uso; estavel
Validacao	Zod	3.x	Ja em uso; integracao nativa com TypeScript
Banco de dados	PostgreSQL	16	RLS nativo; JSONB; pgcrypto
Cache / Fila	Redis	7.x	Rate limiting, cache de QR Code PIX, BullMQ
Jobs / Fila	BullMQ	5.x	Substitui polling manual; persistencia em Redis
Automacao	n8n self-hosted	1.x	Regua de cobranca; integracao WhatsApp e e-mail
Frontend	React 18 + Vite + TypeScript	--	Ja em uso; manter stack
UI Components	Shadcn/ui + Tailwind CSS	--	Design system maduro; acessivel
Gateway PIX/Boleto	Asaas (primario) ou Pagar.me	API v3	Cobranca unica; suporte PIX e boleto registrado
NF-e / NFS-e	Focus NFe API	v2	API REST para emissao; suporte NFS-e municipal
E-mail	Resend	--	API simples; logs nativos
WhatsApp	Z-API ou Twilio	--	Envio de cobranca e confirmacao de pagamento
Monitoramento	Sentry + Grafana	--	Erros e metricas de performance
CI/CD	GitHub Actions	--	Ja configurado; expandir
Deploy	Railway (backend) + Vercel (frontend)	--	PaaS gerenciado; suporte Postgres e Redis

4.2 Estrutura de Diretorios (target)

```
cobranca-saas-api/  
├── src/  
│   ├── modules/  
│   │   ├── billing-core/           # Cobrancas core (existente)  
│   │   ├── identity-access/       # Auth JWT (existente)  
│   │   ├── inbox/                 # Webhook inbox (existente)  
│   │   ├── portal-read/           # Portal CRUD (existente, parcial)  
│   │   ├── tenant-provisioning/   # Onboarding escritorios (existente, parcial)  
│   │   ├── payment-gateway/       # NOVO: Asaas/Pagar.me adapter  
│   │   ├── nfse/                  # NOVO: Emissao NFS-e via Focus NFe  
│   │   ├── notifications/         # NOVO: E-mail (Resend) e WhatsApp (Z-API)  
│   │   ├── reporting/             # NOVO: Relatorios e export CSV  
│   │   └── saas-billing/           # NOVO: Planos, assinaturas, billing  
│   └── platform/  
│       ├── config/                # Runtime flags  
│       ├── health/                # Liveness/readiness  
│       └── http/middleware/        # Auth, RBAC, tenant, rate-limit (Redis)
```

└─ jobs/	# BullMQ workers e schedulers
└─ persistence/	# Pool, RLS, transactions
└─ tenancy/	# Resolucao de tenant
└─ shared/types/	
└─ apps/portal-web/	# SPA React - escritorio (existente)
└─ apps/portal-cliente/	# NOVO: SPA React - cliente final
└─ db/migrations/	# SQL versionado (000-012 existentes)
└─ docs/	# Contratos e runbooks
└─ tests/	# Integracao e funcional
└─ infra/	# Dockerfile, docker-compose, IaC

4.3 Middleware Chain Completo (ordem de execucao)

23. correlationId -- gera x-correlation-id unico por request (implementado).
24. httpAccessLog -- log JSON estruturado (implementado).
25. rateLimiter (NOVO) -- Redis-backed via express-rate-limit:
 - Auth login: 10 req/min por IP.
 - Webhooks inbox: 100 req/min por tenant.
 - Portal CRUD: 300 req/min por usuario autenticado.
26. optionalCors -- CORS baseado em CORS_ORIGIN (implementado).
27. tenantResolution -- resolve tenant_id por header ou claim JWT (implementado).
28. authJwt -- valida JWT; injeta ctx.userId, ctx.tenantId, ctx.roles (implementado).
29. portalMembership -- valida membership ativa no tenant (implementado).
30. rbac -- verifica papel minimo exigido pela rota (implementado).
31. pgTenantSession -- SET LOCAL app.tenant_id = uuid para RLS (implementado).

4.4 Banco de Dados -- Schema Completo (target)

4.4.1 Novas tabelas -- Schema public

Tabela	Proposito	Colunas principais
payment_transactions	Registro de cada tentativa de cobranca junto ao gateway	id UUID PK, charge_id FK, gateway (asaas/pagarme), gateway_transaction_id TEXT UNIQUE, type (boleto/pix), status, amount NUMERIC(14,2), boleto_url, boleto_barcode, boleto_pdf_url, pix_qrcode_base64, pix_emv TEXT, pix_link, expires_at TIMESTAMPTZ, created_at, updated_at
communication_events	Log de cada comunicacao enviada	id UUID PK, tenant_id, charge_id FK, channel (email/whatsapp/sms), event_type (lembrete_pre_3d/pre_1d/vencimento/pos_3d/pos_7d/pago), recipient TEXT, status (pending/sent/failed/delivered), sent_at TIMESTAMPTZ, provider_message_id TEXT, error_message TEXT, created_at
notification_templates	Templates de mensagem por tenant e evento	id UUID PK, tenant_id, event_type, channel, subject TEXT, body_template TEXT (Handlebars), is_active BOOLEAN, created_at, updated_at
charging_rules	Configuracao da regua por tenant	id UUID PK, tenant_id, days_offset INT (negativo=antes, positivo=depois), channel, template_id FK, is_active BOOLEAN, created_at
nfse_emissions	Registro de emissoes NFS-e por cobranca	id UUID PK, tenant_id, charge_id FK UNIQUE, provider (focus_nfe), external_ref TEXT, numero_nfse TEXT, codigo_verificacao TEXT, pdf_url TEXT, xml_url TEXT,

Tabela	Propósito	Colunas principais
		status (emitindo/autorizado/erro/cancelado), emitted_at TIMESTAMPTZ, error_message TEXT, created_at
escritorio_config	Configurações fiscais e de gateway por escritório	id UUID PK, tenant_id TEXT UNIQUE, cnpj_emissor TEXT, razao_social TEXT, inscricao_municipal TEXT, regime_tributario (simples/presumido/real), codigo_municipio TEXT, aliquota_iss NUMERIC(5,2), gateway_provider (asaas/pagarme), gateway_api_key_encrypted TEXT, focus_nfe_token_encrypted TEXT, whatsapp_provider (zapi/twilio), whatsapp_token_encrypted TEXT, encryption_iv TEXT, created_at, updated_at
planos	Catálogo de planos SaaS	id UUID PK, nome TEXT, slug TEXT UNIQUE, max_clientes INT, max_cobrancas_mes INT, preco_mensal NUMERIC(10,2), preco_anual NUMERIC(10,2), features JSONB, is_active BOOLEAN, created_at
assinaturas	Assinatura do escritório em um plano	id UUID PK, tenant_id TEXT UNIQUE, plano_id FK, status (trial/active/past_due/suspended/canceled), trial_ends_at TIMESTAMPTZ, current_period_start TIMESTAMPTZ, current_period_end TIMESTAMPTZ, gateway_subscription_id TEXT, cancel_at_period_end BOOLEAN DEFAULT false, created_at, updated_at
audit_log	Log de auditoria de todas as mutações	id UUID PK, tenant_id TEXT, user_id TEXT, action TEXT (create/update/delete/cancel), resource_type TEXT, resource_id TEXT, old_value JSONB, new_value JSONB, ip TEXT, user_agent TEXT, created_at

4.4.2 Novas tabelas -- Schema portal

Tabela	Propósito	Colunas principais
portal.cliente	Dados completos do cliente final	id UUID PK, tenant_id TEXT, documento TEXT, tipo_documento (cpf/cnpj), nome TEXT, email TEXT, telefone TEXT, opt_in_email BOOLEAN DEFAULT true, opt_in_whatsapp BOOLEAN DEFAULT false, metadata JSONB DEFAULT {}, created_at, updated_at. UNIQUE (tenant_id, documento)
portal.billing_tenant_link	Vínculo entre tenant do portal e tenant público do billing	id UUID PK, portal_tenant_id TEXT, public_tenant_id UUID FK tenants(id), created_at. Já parcialmente implementado.

4.5 Módulo payment-gateway (NOVO -- core do produto)

4.5.1 Interface e Abstração

Para evitar lock-in com um único gateway, o módulo expõe uma interface comum implementada por cada adapter:

```
interface PaymentGatewayAdapter {
  createCustomer(input: CreateCustomerInput): Promise<string>; // retorna gatewayCustomerId
  createBoleto(input: CreateBoletoInput): Promise<BoletoResult>;
  createPix(input: CreatePixInput): Promise<PixResult>;
  cancelCharge(gatewayId: string): Promise<void>;
  getCharge(gatewayId: string): Promise<GatewayChargeStatus>;
}
```

```
// Implementacoes:
// - AsaasAdapter (primario -- Fase 1)
// - PagarmeAdapter (fallback / migracao futura)
```

4.5.2 Fluxo Completo de Emissao de Boleto

32. Operador chama POST /v1/portal/cobrancas com type=boleto.
33. createPortalCharge cria registro charges com canonical_status=rascunho e payment_transactions vazia.
34. Job BullMQ payment-emission enfileirado com delay 0ms.
35. Worker busca dados do cliente no portal.cliente e as configuracoes do gateway em escritorio_config.
36. Worker chama AsaasAdapter.createCustomer() se cliente ainda nao tiver gateway_customer_id.
37. Worker chama AsaasAdapter.createBoleto() com dados do cliente e cobranca.
38. Asaas retorna: boleto_url, linha_digitavel, codigo_barras, pdf_url, nosso_numero.
39. Worker salva resultado em payment_transactions e atualiza charges.canonical_status=emitida e charges.provider_charge_id.
40. Worker de notificacao enfileira envio de e-mail e/ou WhatsApp com link do boleto para o cliente.
41. Webhook Asaas evento PAYMENT_CONFIRMED chega em POST /v1/inbox/webhooks.
42. InboxProcessor processa webhook: charges.canonical_status=paga, communication_events registrado.
43. Evento charge.paid publicado na fila BullMQ para disparo de NFS-e.

4.5.3 Fluxo Completo de Emissao de PIX

44. Operador chama POST /v1/portal/cobrancas com type=pix.
45. createPortalCharge cria registro com canonical_status=rascunho.
46. Job BullMQ payment-emission enfileirado.
47. Worker chama AsaasAdapter.createPix() -- gera cobranca PIX dinamica (Pix Cobranca padrao BACEN).
48. Asaas retorna: pix_qrcode_base64 (imagem PNG), pix_emv (payload EMV para copiar e colar), pix_link.
49. Worker salva em payment_transactions; charges.canonical_status=emitida.
50. Frontend busca payment_transactions e exibe QR Code na tela de detalhe da cobranca.
51. Frontend faz polling de GET /v1/portal/cobrancas/:id a cada 5 segundos por ate 5 minutos apos exibicao do QR Code.
52. Webhook Asaas PAYMENT_CONFIRMED: charges.canonical_status=paga; frontend atualiza tela para "Pago".
53. Disparo de NFS-e conforme Epico 6.

4.5.4 Payload de Webhook Asaas (entrada no /v1/inbox/webhooks)

```
// Header: X-Webhook-Secret: {WEBHOOK_INBOX_SECRET}
// Header: x-tenant-id: {public_tenant_id}
{
  "event": "PAYMENT_CONFIRMED",
  "payment": {
    "id": "pay_xxx",
    "customer": "cus_xxx",
    "value": 1500.00,
```

```
"netValue": 1482.00,  
"status": "CONFIRMED",  
"paymentDate": "2026-05-17",  
"billingType": "PIX",  
"externalReference": "{idempotency_key_da_charge}"  
}  
}
```

4.5.5 Mapeamento Asaas Event -> canonical_status

Evento Asaas	canonical_status resultante	Acao adicional
PAYMENT_CONFIRMED	paga	Dispara emissao de NFS-e via job nfse-emit
PAYMENT_RECEIVED	paga	Equivalente ao CONFIRMED para boleto compensado
PAYMENT_OVERDUE	vencida	Dispara regua pos-vencimento (D+1, D+3, D+7)
PAYMENT_DELETED	cancelada	Cancela jobs de notificacao pendentes
PAYMENT_RESTORED	emitida	Reativa cobranca cancelada no gateway
PAYMENT_REFUNDED	cancelada	Estorno confirmado; notifica cliente

4.6 Integracao Asaas API v3 -- Chamadas Detalhadas

4.6.1 Autenticacao

```
// Header obrigatorio em todas as chamadas:  
access_token: {ASAAS_API_KEY}  
Content-Type: application/json  
  
// Ambientes:  
// Sandbox: https://sandbox.asaas.com/api/v3  
// Producao: https://api.asaas.com/api/v3
```

4.6.2 Criar Cliente no Asaas

```
POST /api/v3/customers  
{  
  "name": "Empresa XYZ Ltda",  
  "cpfCnpj": "12345678000195",  
  "email": "financeiro@xyz.com.br",  
  "phone": "11999999999",  
  "externalReference": "{portal_cliente_id}"  
}  
// Resposta: { "id": "cus_xxx", "name": "...", "cpfCnpj": "...", ... }
```

4.6.3 Criar Cobranca Boleto

```
POST /api/v3/payments  
{  
  "customer": "cus_xxx",  
  "billingType": "BOLETO",  
  "value": 1500.00,  
  "dueDate": "2026-06-10",  
  "description": "Mensalidade Contabilidade - Junho/2026",  
}
```

```
"externalReference": "{idempotency_key}",
"fine": { "value": 2 },           // 2% de multa
"interest": { "value": 0.033 },  // ~1% a.m. = 0.033% ao dia
"discount": { "value": 0 },
"postalService": false
}
// Resposta inclui: bankSlipUrl, identificationField, barCode, nossoNumero
```

4.6.4 Criar Cobrança PIX

```
POST /api/v3/payments
{
  "customer": "cus_xxx",
  "billingType": "PIX",
  "value": 1500.00,
  "dueDate": "2026-06-10",
  "description": "Mensalidade Contabilidade - Junho/2026",
  "externalReference": "{idempotency_key}"
}
// Resposta inclui: pixTransaction.encodedImage (QR base64), payload (EMV string)
// Para buscar QR Code: GET /api/v3/payments/{id}/pixQrCode
```

4.6.5 Cancelar Cobrança

```
DELETE /api/v3/payments/{gateway_transaction_id}
// Resposta: { "deleted": true }
// Webhook PAYMENT_DELETED será disparado em seguida
```

4.6.6 Consultar Status de Cobrança

```
GET /api/v3/payments/{gateway_transaction_id}
// Usado pelo job charge-status-sync para reconciliação de status
// quando webhooks não chegarem (fallback de segurança)
```

4.6.7 Configurar Webhook no Asaas

```
POST /api/v3/webhook
{
  "url": "https://api.suaempresa.com.br/v1/inbox/webhooks",
  "email": "ops@suaempresa.com.br",
  "apiVersion": 3,
  "authToken": "{WEBHOOK_INBOX_SECRET}",
  "updatePending": true,
  "events": [
    "PAYMENT_CONFIRMED", "PAYMENT_RECEIVED",
    "PAYMENT_OVERDUE", "PAYMENT_DELETED",
    "PAYMENT_RESTORED", "PAYMENT_REFUNDED"
  ]
}
```

4.7 Integração PIX -- Padrão BACEN (API Pix v2.2)

Para implementações diretas via API de bancos (Banco Inter, Sicredi, Bradesco, etc.) sem intermediário, o padrão PIX segue a especificação OpenAPI do Banco Central (BACEN). Os endpoints principais são:

Endpoint BACEN	Método	Descrição	Auth
/v2/cob/{txid}	PUT	Cria cobrança PIX imediata (QR Code dinâmico sem vencimento)	mTLS + OAuth2 client_credentials
/v2/cobv/{txid}	PUT	Cria cobrança PIX com vencimento (substitui boleto)	mTLS + OAuth2
/v2/cobv/{txid}	GET	Consulta cobrança e verifica se foi paga	mTLS + OAuth2
/v2/pix/{e2eld}	GET	Consulta Pix recebido pelo identificador de transação E2E	mTLS + OAuth2
/v2/webhook	PUT	Configura URL de notificação de recebimento	mTLS + OAuth2
/v2/pix	GET	Lista Pix recebidos por período (para reconciliação)	mTLS + OAuth2

4.7.1 Autenticação PIX BACEN (OAuth2 + mTLS)

```
// 1. Gerar certificado e-CNPJ A1 ou A3 (obrigatório pelo BACEN)
// 2. POST no endpoint OAuth2 do banco:
POST https://{banco}/oauth/token
Content-Type: application/x-www-form-urlencoded
// Com certificado mTLS na requisição TLS
grant_type=client_credentials
&client_id={client_id}
&client_secret={client_secret}
&scope=cob.write cob.read cobv.write cobv.read pix.read webhook.write webhook.read

// Resposta: { access_token, token_type: "Bearer", expires_in: 3600 }
// O access_token é usado no header Authorization das chamadas subsequentes
```

4.7.2 Payload de Criação de Cobrança PIX com Vencimento (cobv)

```
PUT /v2/cobv/{txid}
Authorization: Bearer {access_token}
{
  "calendario": {
    "dataDeVencimento": "2026-06-10",
    "validadeAposVencimento": 30
  },
  "devedor": {
    "cnpj": "12345678000195", // ou "cpf" para pessoa física
    "nome": "Empresa XYZ Ltda"
  },
  "valor": {
    "original": "1500.00",
    "multa": { "modalidade": 2, "valorPerc": "2.00" },
    "juros": { "modalidade": 2, "valorPerc": "1.00" },
    "desconto": { "modalidade": 0 }
  },
  "chave": "suachave@empresa.com.br", // chave PIX do recebedor
  "solicitacaoPagador": "Mensalidade Contabilidade - Junho/2026",
  "infoAdicionais": [
    { "nome": "reference", "valor": "{idempotency_key}" }
  ]
}
```

4.7.3 Webhook de Notificação PIX (BACEN -> sistema)

```
// O BACEN envia POST para a URL configurada quando um PIX é recebido:
{
  "pix": [{
    "endToEndId": "E12345678202605171200xxxxx",
    "txid": "{txid_da_cobranca}",
    "valor": "1500.00",
    "horario": "2026-05-17T12:00:00.000Z",
    "pagador": {
      "cpf": "12345678909",
      "nome": "Joao da Silva"
    },
    "infoPagador": "Mensalidade"
  ]
}
```

NOTA: A integração direta com a API BACEN (por banco) é mais complexa e exige certificado digital e-CNPJ. O uso do Asaas como intermediário (Fase 1) abstrai toda essa complexidade. A integração direta é recomendada apenas em Fase 3+ quando o volume justificar os custos operacionais de manutenção.

4.8 Modulo nfse -- NFS-e via Focus NFe

4.8.1 Fluxo de Emissao

54. Evento paga processado pelo InboxProcessor.
55. InboxProcessor publica evento interno charge.paid na fila BullMQ.
56. Worker nfse-emit consome o evento.
57. Busca escritorio_config do tenant para obter CNPJ emissor, inscricao municipal, regime tributario, aliquota ISS e token Focus NFe (criptografado em repouso; decryptografado em memoria).
58. Busca dados do tomador no portal.cliente (CNPJ, razao social, endereco).
59. Monta payload da NFS-e conforme regras do municipio do prestador.
60. POST /v2/nfse?ref={charge_id} na API Focus NFe.
61. Focus NFe retorna status="emitindo"; worker registra em nfse_emissions com status=emitindo.
62. Job nfse-poll faz polling a cada 10 segundos por ate 5 minutos.
63. Quando Focus retorna status="autorizado": salva numero_nfse, pdf_url, xml_url; atualiza charges.metadata.nfse_id.
64. Worker notifications-send envia e-mail ao cliente com PDF da NFS-e em anexo.

4.8.2 Payload Focus NFe -- NFS-e

```
POST https://api.focusnfe.com.br/v2/nfse?ref={charge_id}
Authorization: Token {FOCUS_NFE_TOKEN}
Content-Type: application/json
{
  "data_emissao": "2026-05-17",
  "prestador": {
    "cnpj": "12345678000195",
    "inscricao_municipal": "12345",
    "codigo_municipio": "3550308" // Sao Paulo = 3550308
  },
  "tomador": {
    "cnpj": "98765432000112",
    "razao_social": "Empresa XYZ Ltda",
    "email": "financeiro@xyz.com.br",
    "telefone": "1133333333",
  }
}
```

```
"endereco": {
  "logradouro": "Rua das Flores",
  "numero": "100",
  "complemento": "Sala 10",
  "bairro": "Centro",
  "codigo_municipio": "3550308",
  "uf": "SP",
  "cep": "01310-100"
},
"servico": {
  "aliquota": 5.00,
  "base_calculo": 1500.00,
  "discriminacao": "Servicos contabeis - Junho/2026. Referente a cobranca {charge_reference}.",
  "iss_retido": false,
  "codigo_tributario_municipio": "01.01",
  "valor_servicos": 1500.00,
  "item_lista_servico": "17.20" // LC 116/2003
}
}

// Consulta de status (polling):
GET https://api.focusnfe.com.br/v2/nfse?ref={charge_id}
// Resposta: { "status": "autorizado", "numero": "12345", "codigo_verificacao": "ABCDE",
"url": "...", "caminho_xml_nota_fiscal": "..." }
```

4.9 Modulo notifications -- E-mail e WhatsApp

4.9.1 E-mail via Resend

```
POST https://api.resend.com/emails
Authorization: Bearer {RESEND_API_KEY}
Content-Type: application/json
{
  "from": "cobrancas@suaempresa.com.br",
  "to": ["financeiro@cliente.com.br"],
  "subject": "Cobranca vence em 3 dias -- R$ 1.500,00",
  "html": "... template Handlebars renderizado ...",
  "attachments": [
    {
      "filename": "boleto.pdf",
      "content": "{base64 do PDF}"
    }
  ]
}
```

4.9.2 WhatsApp via Z-API

```
POST https://api.z-api.io/instances/{instance}/token/{token}/send-text
Content-Type: application/json
{
  "phone": "5511999999999",
  "message": "Ola Joao, seu boleto de R$ 1.500,00 vence em 3 dias.\nLink para pagar: {link_boleto}\nOu pague via PIX: {link_pix}",
  "delayMessage": 2
}
```

4.9.3 Regua de Cobrança -- Templates Padrão

Trigger	Canal	Assunto / Mensagem padrão	Variáveis disponíveis
D-3 (3 dias antes)	WhatsApp + E-mail	[Lembrete] Seu boleto de {{valor}} vence em {{data_vencimento}}	{{nome}}, {{valor}}, {{data_vencimento}}, {{link_boleto}}, {{link_pix}}, {{escritorio_nome}}
D-1 (vespera)	WhatsApp	Último lembrete: boleto vence amanhã ({{data_vencimento}}). Pague com PIX: {{link_pix}}	Idem acima
D+0 (vencimento)	E-mail	[Aviso] Seu boleto venceu hoje -- R\$ {{valor}}	Idem + {{multa_percentual}}, {{juros_dia}}
D+3	WhatsApp + E-mail	Boleto vencido há 3 dias. Multa e juros aplicados. 2ª via: {{link_2a_via}}	Idem + {{valor_atualizado}}
D+7	E-mail	Último aviso antes de negativacao. Regularize em até 48h.	Idem
Pago	E-mail	Pagamento confirmado! NFS-e em anexo.	{{nome}}, {{valor}}, {{data_pagamento}}, {{numero_nfse}}

4.10 Jobs BullMQ (substitui polling manual)

Job	Fila Redis	Trigger	Retry	Descrição
payment-emission	charges:emission	POST cobrança (delay 0)	3x backoff 30s	Emitir boleto/PIX no gateway via adapter.
webhook-process	inbox:process	POST /inbox/webhooks	3x backoff 60s	Processa evento do inbox; atualiza canonical_status.
charge-status-sync	charges:sync	Cron cada 15min	--	Consulta status de cobranças emitidas há >24h sem webhook.
notification-send	notifications:send	Charge event ou cron regua	3x backoff 120s	Envia e-mail ou WhatsApp via template.
nfse-emit	nfse:emit	charge.paid event (delay 5s)	5x backoff 60s	Emitir NFS-e via Focus NFe.
nfse-poll	nfse:poll	Após nfse-emit (10s delay)	10x backoff 10s	Polling de status emitindo->autorizado.
subscription-renew	billing:renew	Cron diário 08h	2x backoff 300s	Renova assinaturas; downgrade se falhar.

5. Frontend -- Especificacao de Telas

5.1 Portal do Escritorio (apps/portal-web)

Rota	Tela	Status atual	Spec completa
/login	Login	Implementado	E-mail + senha + tenant_id. Redirect para dashboard apos token. Mensagem de erro generica (sem distinguir e-mail de senha por seguranca).
/dashboard	Dashboard	Implementado (parcial)	Cards: recebido no mes, em aberto, vencidos, taxa inadimplencia. Grafico line chart 12 meses. Top 5 clientes inadimplentes. Proximos 7 vencimentos.
/clientes	Lista clientes	Implementado	Tabela cursor paginacao. Busca por nome/documento. Filtro status cobranca. Botao novo cliente.
/clientes/novo	Cadastro cliente	Implementado	Form: documento (CPF/CNPJ + mascara + validacao), nome, e-mail, telefone, opt-ins. Validacao inline.
/clientes/:id	Detalhe cliente	Implementado	Dados cadastrais + lista cobrancas com status chips + botao nova cobranca.
/clientes/:id/editar	Editar cliente	Implementado	PATCH nome/e-mail/opt-in. Documento read-only com tooltip explicativo.
/cobrancas	Lista cobrancas	Implementado	Tabela paginada. Filtro: status, periodo, cliente. Botao export CSV.
/cobrancas/nova	Nova cobranca	Parcial	Selecionar cliente, tipo (boleto/PIX), valor, vencimento, descricao. NOVO: opcao de recorrancia (mensal/trimestral/anual).
/cobrancas/:id	Detalhe cobranca	Parcial	Status chip + historico de eventos + QR Code PIX (polling 5s) + link boleto/PDF + botao cancelar + link NFS-e (quando disponivel).
/relatorios	Relatorios	PLACEHOLDER	Filtros: periodo, status, cliente. Cards de totais (recebido, pendente, vencido, inadimplencia%). Grafico barras mensal. Export CSV e Excel.
/notas-fiscais	Notas Fiscais	Parcial	Lista NFS-e emitidas. Download PDF e XML. Filtro por periodo e status.
/escritorio	Config escritorio	Parcial	CNPJ emissor, dados fiscais (ISS, inscricao municipal), API key gateway (mascarada), configuracao da regua de cobranca.
/usuarios	Usuarios	NOVO	Lista memberships ativas. Convidar operador por e-mail. Desativar acesso. Historico de convites.
/plano	Plano e assinatura	NOVO	Plano atual, uso do mes (clientes, cobrancas), upgrade/downgrade, historico de faturas com download.

5.2 Portal do Cliente Final (apps/portal-cliente) -- NOVO

Rota	Tela	Descricao completa
/login	Login cliente	E-mail + senha. Subdominio ou query param ?escritorio={slug} resolve o tenant. Sem exposicao do tenant_id para o usuario final.

Rota	Tela	Descricao completa
/minhas-cobrancas	Minhas cobrancas	Lista cobrancas em aberto e pagas. Filtro status. Cards de: total em aberto, vencidas, pagas no mes. QR Code PIX inline no card da cobranca.
/minhas-cobrancas/:id	Detalhe cobranca	QR Code PIX (com countdown de expiracao) ou link para boleto PDF. Polling automatico de pagamento. Download NFS-e quando emitida. Historico de tentativas de pagamento.
/historico	Historico	Cobrancas pagas com NFS-e disponivel para download. Filtro por periodo.
/perfil	Meu perfil	Alterar senha. Configurar preferencias de notificacao (opt-in WhatsApp/e-mail).

5.3 Componentes de UI Prioritarios (Shadcn/ui + Tailwind)

- ChargeStatusBadge -- chip colorido por canonical_status: verde (paga), azul (emitida/enviada), amarelo (pendente_pagamento), vermelho (vencida), cinza (rascunho/cancelada).
- CurrencyInput -- input formatado R\$ com mascara BRL em tempo real.
- DocumentInput -- CPF/CNPJ com mascara automatica e validacao de digito verificador no frontend (sem round-trip ao servidor para validacao basica).
- DatePicker -- seletor de data com bloqueio de datas passadas para vencimento.
- PixQrCode -- componente que exhibe QR Code base64, countdown de expiracao (15 min padrao) e polling de status com spinner.
- CursorTable -- tabela com paginacao por cursor, loading skeleton e botao "Carregar mais" no rodape.
- ExportCsvButton -- dispara download de CSV via streaming do backend sem bloquear a UI.
- NotificationTemplateEditor -- textarea com preview das variaveis disponiveis e contador de caracteres (para WhatsApp 1024 chars).
- AuditTimeline -- lista cronologica de eventos de uma cobranca (emitida, enviada, visualizada, paga, NFS-e emitida).

6. Segurança e Compliance

6.1 Gestão de Secrets

- Nunca commitar .env com valores reais. Usar .env.example com placeholders descritivos.
- Em produção: Railway Variables (PaaS simples), AWS Parameter Store + Secrets Manager (enterprise), ou HashiCorp Vault (on-premise).
- Chaves de API de gateway (Asaas, Focus NFe, Z-API) armazenadas criptografadas em `escritorio_config` usando AES-256-GCM com `ENCRYPTION_KEY` derivada de KMS ou variável de ambiente.
- `JWT_SECRET` mínimo 64 bytes aleatórios (`openssl rand -base64 64`). Rotação trimestral com janela de transição de 1h (aceitar token assinado com key antiga durante a janela).
- `WEBHOOK_INBOX_SECRET` distinto do `JWT_SECRET`. Rotação independente com atualização sincronizada no Asaas.

6.2 Rate Limiting (Redis-backed)

Rota	Limite	Janela	Chave de limite	Resposta ao exceder
POST /v1/portal/auth/login	10 req	1 minuto	Por IP	429 + Retry-After header
POST /v1/tenants/provision	5 req	1 hora	Por IP	429 + mensagem de suporte
POST /v1/inbox/webhooks	100 req	1 minuto	Por x-tenant-id	429 + Retry-After: 60
POST /v1/portal/clientes	200 req	1 minuto	Por user_id autenticado	429
GET/* (portal autenticado)	300 req	1 minuto	Por user_id	429

6.3 LGPD -- Dados Sensíveis

- Dados de clientes (CPF, CNPJ, e-mail) nunca aparecem em logs de acesso -- usar `correlation_id` para rastrear requests.
- Data retention: cobranças e dados financeiros retidos por 5 anos (obrigação fiscal Lei 6.404/1976); dados de comunicação por 1 ano.
- Endpoint de exclusão de cliente: soft delete com anonimização de dados pessoais (nome substituído por "CLIENTE EXCLUÍDO", e-mail por hash SHA-256, CPF/CNPJ mantido apenas para obrigação fiscal).
- Consentimento de opt-in explicitamente registrado com timestamp para WhatsApp e e-mail.
- Política de privacidade e LGPD exibidas no cadastro do cliente final.

6.4 Auditoria

Tabela `audit_log` registra todas as mutações em entidades críticas:

```
-- DDL da tabela de auditoria
CREATE TABLE public.audit_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id TEXT NOT NULL,
  user_id TEXT,
```

```
    action TEXT NOT NULL, -- create | update | delete | cancel | status_change
    resource_type TEXT NOT NULL, -- charge | cliente | membership | escritorio_config
    resource_id TEXT NOT NULL,
    old_value JSONB,
    new_value JSONB,
    ip_address TEXT,
    user_agent TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_audit_log_tenant_resource ON audit_log(tenant_id, resource_type,
resource_id);
CREATE INDEX idx_audit_log_tenant_created ON audit_log(tenant_id, created_at DESC);
```

7. Roadmap de Desenvolvimento

7.1 Fase 0 -- Saneamento (2 semanas, pre-requisito)

- 65. Remover .env do repositório; adicionar ao .gitignore; rotacionar todos os secrets expostos no zip.
- 66. Desacoplar schema automacao: criar tabelas notas_fiscais e fiscal_tenants no banco proprio da aplicacao; escrever script de migracao/sincronizacao se dados existirem.
- 67. Cobrir provision-public-tenant.ts com testes de integracao (de 45% para >= 85%).
- 68. Adicionar Dockerfile multi-stage e docker-compose.yml com PostgreSQL, Redis e API.
- 69. Configurar Sentry DSN no backend e no frontend.
- 70. Criar tabela audit_log e middleware de auditoria.

7.2 Fase 1 -- Integracao com Gateway (6 semanas)

- 71. Modulo payment-gateway com AsaasAdapter (Boleto e PIX).
- 72. Tabelas payment_transactions e escritorio_config; migracoes SQL.
- 73. Tabela portal.cliente (desacopla do modelo anterior).
- 74. Jobs BullMQ: payment-emission, webhook-process, charge-status-sync.
- 75. Redis: setup de pool e rate limiting.
- 76. Configuracao de webhook no Asaas via script de setup.
- 77. Mapeamento completo de eventos Asaas -> canonical_status.
- 78. Frontend: tela de detalhe com QR Code PIX e polling de status.
- 79. Frontend: tela nova cobranca com selecao de tipo (boleto/PIX).
- 80. Testes de integracao end-to-end com sandbox Asaas (CI verde).

7.3 Fase 2 -- Notificacoes e Regua (4 semanas)

- 81. Modulo notifications com Resend (e-mail) e Z-API (WhatsApp).
- 82. Tabelas communication_events, notification_templates, charging_rules.
- 83. Job notification-send com BullMQ.
- 84. API de configuracao de regua: CRUD de charging_rules por tenant.
- 85. Tela de configuracao da regua no portal do escritorio.
- 86. Templates padrao editaveis por tenant com preview ao vivo.

7.4 Fase 3 -- NFS-e (3 semanas)

- 87. Modulo nfse com Focus NFe adapter.
- 88. Tabelas nfse_emissions; atualizacao de escritorio_config com dados fiscais.
- 89. Jobs nfse-emit e nfse-poll.
- 90. Tela de configuracao fiscal no portal do escritorio (CNPJ, inscricao municipal, aliquota ISS).
- 91. E-mail automatico com PDF da NFS-e apos emissao.
- 92. Tela de notas fiscais com download PDF e XML.

7.5 Fase 4 -- Relatorios e Portal do Cliente (3 semanas)

- 93. Dashboard: gráfico de recebimentos 12 meses, taxa de inadimplência por cohort.
- 94. Tela de relatórios com filtros avançados e export CSV (streaming).
- 95. SPA portal-cliente (apps/portal-cliente): login, lista cobranças, QR Code, histórico.
- 96. Endpoint de export CSV com streaming (sem carregar tudo em memória).

7.6 Fase 5 -- Billing SaaS e Go-Live (4 semanas)

- 97. Módulo saas-billing: tabelas planos e assinaturas.
- 98. Integração Asaas para cobrar os escritórios (boleto ou cartão recorrente).
- 99. Trial de 14 dias automático ao provisionar novo tenant.
- 100. Downgrade automático para read-only após vencimento sem pagamento.
- 101. Job subscription-renew (cron diário).
- 102. Homologação formal com 3 escritórios piloto (30 dias).
- 103. Go-live com monitoramento Sentry + Grafana + alertas de 5xx e latência p95.

7.7 Estimativa de Custo e Prazo

Fase	Duração	Dev necessário	Custo estimado (senior R\$ 15k/mes)	Entrega
Fase 0 -- Saneamento	2 semanas	1 dev	R\$ 7.500	Base segura e desacoplada
Fase 1 -- Gateway Pgto	6 semanas	1-2 devs	R\$ 22.500 -- R\$ 45.000	Boleto e PIX funcionando
Fase 2 -- Notificações	4 semanas	1 dev	R\$ 15.000	Regua de cobrança automatizada
Fase 3 -- NFS-e	3 semanas	1 dev	R\$ 11.250	NFS-e automática pós-pagamento
Fase 4 -- Relatórios	3 semanas	1 dev	R\$ 11.250	Dashboard e portal do cliente
Fase 5 -- Billing SaaS	4 semanas	1-2 devs	R\$ 15.000 -- R\$ 30.000	Go-live completo
TOTAL	22 semanas (~5 meses)	1-2 devs	R\$ 82.500 -- R\$ 120.000	Produto completo em produção

8. DevOps e Infraestrutura

8.1 Variáveis de Ambiente Completas (target)

Variável	Obrigatório	Descrição
NODE_ENV	Sim	production development test
PORT	Sim	Porta HTTP (default 3333)
DATABASE_URL	Sim	URL Postgres com sslmode=require em produção
REDIS_URL	Sim (Fase 1)	URL Redis para BullMQ e rate limiting
JWT_SECRET	Sim	Mínimo 64 bytes aleatórios (openssl rand -base64 64)
WEBHOOK_INBOX_SECRET	Sim	Secret compartilhado com Asaas (X-Webhook-Secret)
ENCRYPTION_KEY	Sim (Fase 1)	32 bytes para criptografia de API keys de gateway em repouso
ASAAS_API_KEY	Sim (Fase 1)	API key do Asaas (sandbox em dev, produção em prod)
ASAAS_API_URL	Sim (Fase 1)	https://api.asaas.com/api/v3 (ou sandbox)
FOCUS_NFE_TOKEN	Sim (Fase 3)	Token de autenticação Focus NFe
FOCUS_NFE_URL	Sim (Fase 3)	https://api.focusnfe.com.br/v2
RESEND_API_KEY	Sim (Fase 2)	API key Resend para e-mails transacionais
ZAPI_INSTANCE	Sim (Fase 2)	ID da instância Z-API
ZAPI_TOKEN	Sim (Fase 2)	Token de autenticação Z-API
SENTRY_DSN	Recomendado	DSN do projeto Sentry para monitoramento de erros
CORS_ORIGIN	Recomendado	Lista de origens CORS separadas por vírgula
PG_POOL_MAX	Recomendado	Max conexões no pool (default 20)
ENABLE MOCK_AUTH	Não (dev)	true apenas em dev/staging controlado
ALLOW_INSECURE_DATABASE_URL	Não (dev)	1 para Postgres local sem TLS em dev

8.2 Pipeline CI/CD Expandido (GitHub Actions)

```
# .github/workflows/ci.yml (target)
on:
  push: { branches: [main, master, develop] }
  pull_request: { branches: [main, master, develop] }

jobs:
  lint:           # ESLint + TypeScript check (npm run check)
  test:unit:      # Vitest unidade (npm test)
  test:portal:    # Vitest portal-web (npm run portal:test)
  build:          # tsc + vite build (npm run build + npm run portal:build)
  migrate:        # npm run migrate (Postgres de serviço)
  check:readiness: # npm run check:readiness
```

```
test:integration: # npm run test:integration (requer DATABASE_URL)
test:stress:      # npm run test:stress:portal-clientes
security:         # npm audit --audit-level=high + Snyk (opcional)
deploy:staging:   # Deploy para Railway staging (push em develop)
smoke:staging:    # Smoke test /health/ready + criar cobrança de teste
deploy:production: # Deploy para Railway prod (aprovação manual, push em main)
```

8.3 Health Checks e Observabilidade

- GET /health -- liveness probe. Resposta em < 100ms. Retorna { status: ok, service, uptime }.
- GET /health/ready -- readiness probe. Verifica Postgres + Redis + schema minimo. 503 se qualquer check falhar.
- Alertas Sentry: qualquer HTTP 5xx cria issue imediato; unhandled rejections capturadas.
- Grafana/Prometheus: metricas de latencia p50/p95/p99 por rota; taxa de erro; cobranças emitidas/pagas por minuto.
- Alerta: p95 de latencia > 2s nas rotas /v1/portal/* ou /v1/billing/* dispara PagerDuty/Slack.
- Dead Letter Queue no BullMQ: jobs com 3+ falhas movidos para DLQ; alerta no Slack.
- Log JSON estruturado: cada linha inclui correlation_id, tenant_id (mascarado se necessario), method, path, status, duration_ms.

8.4 Checklist de Release para Producao

104. npm run build sem erros.
105. CI verde no GitHub Actions (todos os jobs).
106. Migração aplicada no banco de produção (npm run migrate com DATABASE_URL de prod).
107. npm run check:readiness verde com variáveis de produção.
108. Smoke test manual: GET /health/ready, POST /v1/portal/auth/login, criar cobrança de teste em tenant demo.
109. Verificar que mocks estão desligados: POST /v1/auth/token/mock deve retornar 404.
110. Webhook sem secret retorna 503 (sanity check).
111. CORS configurado para o domínio correto do portal.
112. Sentry DSN configurado e testado (lançar erro de teste e verificar no painel).
113. Rollback plan documentado: tag da versão anterior + script de revert de migrações se necessario.

9. Glossário Técnico

Termo	Definição
canonical_status	Status normalizado da cobrança no sistema, independente de nomenclatura do gateway externo. Mapeado a partir de eventos de webhook.
RLS (Row Level Security)	Recurso do PostgreSQL que filtra automaticamente linhas por uma policy baseada em variável de sessão (app.tenant_id). Garante isolamento de dados entre tenants no nível do banco sem alteração de queries.
Tenant	Escritório ou empresa cadastrada na plataforma. Cada tenant tem seus próprios dados isolados por RLS.
idempotency_key	Chave única por cobrança dentro de um tenant. Garante que retentativas da mesma operação não criem duplicatas (ON CONFLICT DO NOTHING).
Cursor paginação (keyset)	Paginação por ponteiro opaco que evita o problema de offset em grandes conjuntos de dados. Mais eficiente que LIMIT/OFFSET para tabelas com muitos registros.
BullMQ	Biblioteca de filas baseada em Redis para Node.js. Substitui o polling manual; suporta retry, backoff exponencial, delay e dead letter queue.
EMV/Pix	Padrão de payload do QR Code Pix definido pelo Banco Central do Brasil. O campo "payload" retornado pelo Asaas e o EMV string que qualquer app bancário consegue ler.
Focus NFe	API REST brasileira para emissão de NF-e, NFS-e, NFC-e e CT-e. Abstrai as diferenças entre municípios e estados para emissão fiscal.
Asaas	Plataforma brasileira de automação financeira com API para Boleto FEBRABAN, Pix BACEN, cartão de crédito e cobrança recorrente. Elimina a necessidade de certificado bancário para emissão.
e2eld	Identificador fim-a-fim de uma transação Pix no sistema BACEN. Formato: E{ISPB_banco}{data_hora}{sequencial}. Único por transação no SPI.
MRR	Monthly Recurring Revenue -- receita mensal recorrente gerada pelos planos dos escritórios na plataforma SaaS.
DLQ (Dead Letter Queue)	Fila de jobs que falharam todas as tentativas de retry. Jobs na DLQ requerem investigação e reprocessamento manual.
mTLS (Mutual TLS)	Autenticação mútua via certificado X.509 em ambos os lados da conexão TLS. Obrigatório pela API Pix do BACEN.
NFS-e	Nota Fiscal de Serviços eletrônica. Emitida por prestadores de serviço (escritórios contábeis, consultorias). Município-specific: cada prefeitura tem layout e regras próprias, abstraídos pelo Focus NFe.
webhook inbox pattern	Padrão de recebimento de eventos externos: salvar primeiro (inbox), processar depois (job assíncrono). Garante que eventos não se percam em caso de falha de processamento.

10. Referências

- Especificação API Pix BACEN v2.2: <https://bacen.github.io/pix-api>
- Documentação Asaas API v3: <https://docs.asaas.com>
- Focus NFe API Docs: <https://focusnfe.com.br/doc>
- Resend API Docs: <https://resend.com/docs>

- BullMQ Docs: <https://docs.bullmq.io>
- Shadcn/ui: <https://ui.shadcn.com>
- FEBRABAN -- Padrao de Boleto Registrado: <https://febraban.org.br>
- LGPD -- Lei Geral de Protecao de Dados (Lei 13.709/2018): <https://www.gov.br/lgpd>
- LC 116/2003 -- Lista de Servicos NFS-e: https://www.planalto.gov.br/ccivil_03/leis/lc/lc116.htm
- Redis Rate Limiting com express-rate-limit: <https://github.com/express-rate-limit/rate-limit-redis>